

Convolution Neural Network

October 30, 2024

A Convolutional Neural Network (CNN) is a type of deep learning model primarily used for processing structured grid data, such as images, and recognizing patterns. CNNs are widely applied in tasks like image recognition, object detection, video analysis, and natural language processing, thanks to their ability to capture spatial and hierarchical information from the data.

0.0.1 Key Components of CNNs

0.0.2 Convolutional Layers

These layers apply filters (kernels) to the input data to detect features, such as edges, textures, and shapes in an image. The filter slides across the input, performing a convolution operation that produces a feature map, which highlights the presence of certain patterns in specific regions.

0.0.3 Pooling Layers

Pooling layers reduce the spatial dimensions of the feature maps, which decreases the number of parameters, speeds up computation, and helps control overfitting. The most common pooling method is max pooling, which takes the maximum value in a region, retaining only the most significant feature.

0.0.4 Fully Connected Layers

After a series of convolutional and pooling layers, the output is often flattened and passed through fully connected (dense) layers. These layers combine all features detected by the convolutional layers to make final predictions.

0.0.5 Activation Functions

Activation functions introduce non-linearity into the network, allowing it to learn complex patterns. ReLU (Rectified Linear Unit) is the most common activation function in CNNs, setting all negative values to zero to enhance computational efficiency.

0.0.6 Dropout (Optional)

Dropout is a regularization technique that randomly “drops” (sets to zero) a fraction of the neurons during training, which helps prevent overfitting by ensuring that the network does not rely too heavily on specific neurons.

0.0.7 How CNNs Work for Image Classification

In an image classification task, a CNN might take an image as input and perform the following steps:

1. Extract Low-Level Features: The first layers typically detect simple patterns, like edges and corners.
2. Detect Mid-Level Features: As data passes through deeper layers, the model learns more complex features, like shapes and textures.
3. Recognize High-Level Features: In the final layers, the network learns abstract concepts (e.g., parts of objects) and combines them to classify the image.

0.0.8 Advantages of CNNs

1. Translation Invariance: CNNs can recognize patterns regardless of their position within the image.
2. Parameter Efficiency: By using shared weights (filters), CNNs require fewer parameters than fully connected neural networks, making them more computationally efficient.
3. Hierarchical Feature Learning: CNNs learn from simple to complex features, capturing intricate structures within data.

0.0.9 Applications of CNNs

1. Image and Video Analysis: Face recognition, object detection, medical image analysis (e.g., MRI scans).
2. Natural Language Processing: Text classification, sentiment analysis, and language translation.
3. Self-Driving Cars: Detecting objects like pedestrians, road signs, and other vehicles.
4. Healthcare: Diagnosing diseases from medical images, analyzing x-rays, and automating pathology.
5. CNNs are a powerful tool in the deep learning field, especially for tasks involving images or spatial data, due to their ability to understand intricate patterns.

```
[ ]: # importing the necessary libraries

import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns

import warnings
warnings.filterwarnings("ignore")
```

```
[2]: # load the train data
train = pd.read_csv("train.csv")
train.head()
```

```
[2]:
```

	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	\
0	1	0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	0	0	
2	1	0	0	0	0	0	0	0	0	

3	4	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0

	pixel8	...	pixel774	pixel775	pixel776	pixel777	pixel778	pixel779	\
0	0	...	0	0	0	0	0	0	
1	0	...	0	0	0	0	0	0	
2	0	...	0	0	0	0	0	0	
3	0	...	0	0	0	0	0	0	
4	0	...	0	0	0	0	0	0	

	pixel780	pixel781	pixel782	pixel783
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	0

[5 rows x 785 columns]

```
[3]: train.shape
```

```
[3]: (42000, 785)
```

```
[4]: train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 42000 entries, 0 to 41999
Columns: 785 entries, label to pixel783
dtypes: int64(785)
memory usage: 251.5 MB
```

```
[5]: test = pd.read_csv('test.csv')
test.head()
```

```
[5]:
```

	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	\
0	0	0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	0	0	
2	0	0	0	0	0	0	0	0	0	
3	0	0	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	0	0	

	pixel19	...	pixel774	pixel775	pixel776	pixel777	pixel778	pixel779	\
0	0	...	0	0	0	0	0	0	
1	0	...	0	0	0	0	0	0	
2	0	...	0	0	0	0	0	0	
3	0	...	0	0	0	0	0	0	
4	0	...	0	0	0	0	0	0	

	pixel780	pixel781	pixel782	pixel783
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	0

[5 rows x 784 columns]

```
[6]: test.shape
```

```
[6]: (28000, 784)
```

```
[7]: test.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28000 entries, 0 to 27999
Columns: 784 entries, pixel0 to pixel783
dtypes: int64(784)
memory usage: 167.5 MB
```

```
[8]: # put the labels columns as y_train
```

```
y_train = train['label']
X_train = train.drop(labels = ['label'], axis = 1)
```

```
[9]: y_train.head()
```

```
[9]: 0    1
     1    0
     2    1
     3    4
     4    0
     Name: label, dtype: int64
```

```
[10]: X_train.head()
```

```
[10]:
```

	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	\
0	0	0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	0	0	
2	0	0	0	0	0	0	0	0	0	
3	0	0	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	0	0	

	pixel19	...	pixel774	pixel775	pixel776	pixel777	pixel778	pixel779	\
0	0	...	0	0	0	0	0	0	

1	0	...	0	0	0	0	0	0
2	0	...	0	0	0	0	0	0
3	0	...	0	0	0	0	0	0
4	0	...	0	0	0	0	0	0

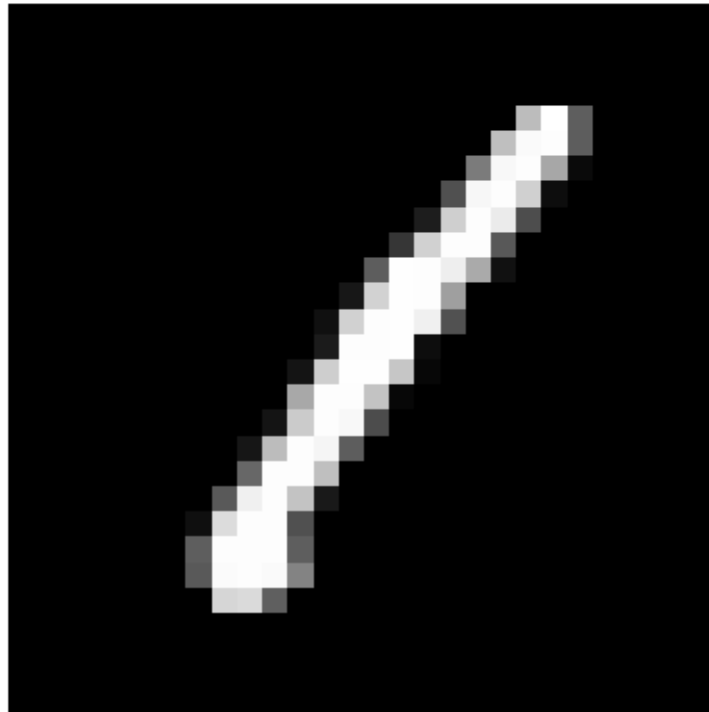
	pixel780	pixel781	pixel782	pixel783
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	0

[5 rows x 784 columns]

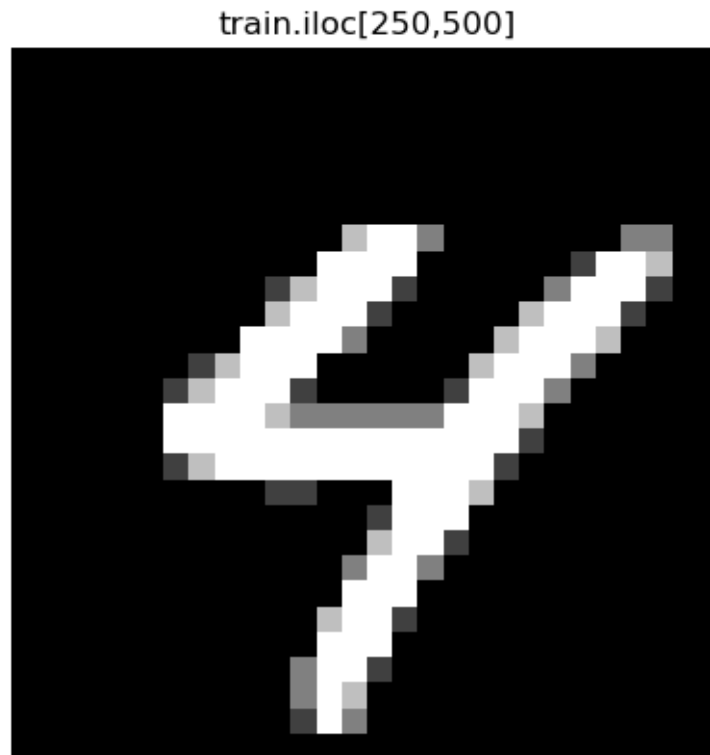
[11]: *#plot some samples in the train*

```
img = X_train.iloc[0].values
img = img.reshape((28,28))
plt.imshow(img, cmap = 'gray')
plt.title('train.iloc[0,0]')
plt.axis('off')
plt.show()
```

train.iloc[0,0]



```
[12]: img = X_train.iloc[250].values
img = img.reshape((28,28))
plt.imshow(img, cmap = 'gray')
plt.title('train.iloc[250,500]')
plt.axis('off')
plt.show()
```



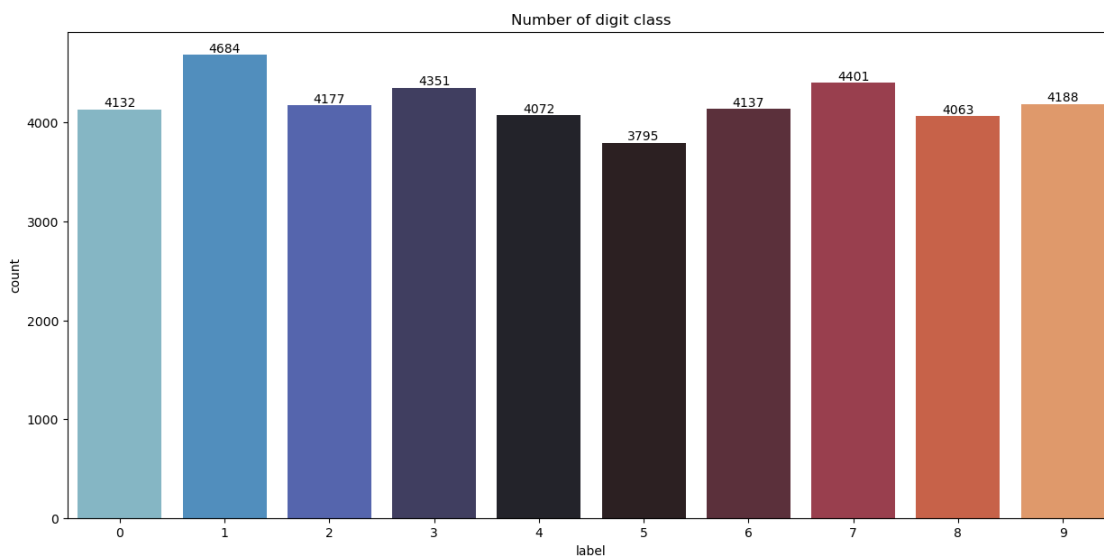
```
[13]: y_train.value_counts()
```

```
[13]: 1    4684
      7    4401
      3    4351
      9    4188
      2    4177
      6    4137
      0    4132
      4    4072
      8    4063
      5    3795
      Name: label, dtype: int64
```

```
[14]: # plot the number of digits in the label columns

plt.figure(figsize = (15,7))
g = sns.countplot(x = y_train, palette = 'icefire')

# add labels to the bar
for patch in g.patches:
    height = patch.get_height()
    g.text(patch.get_x() + patch.get_width()/2, height + 0.5, int(height), ha = 'center', va = 'bottom')
plt.title("Number of digit class")
plt.show()
```



```
[15]: # check for missing values in any rows

missing_values = train.isnull().any(axis=1)
count_missing_values = missing_values.sum()
count_missing_values
```

[15]: 0

```
[16]: missing_values = test.isnull().any(axis=1)
count_missing_values = missing_values.sum()
count_missing_values
```

[16]: 0

```
[17]: # Both train and test does not have any missing values
# Normalize the image data
X_train = X_train/255.0
test = test/255.0

X_train.shape, test.shape
```

```
[17]: ((42000, 784), (28000, 784))
```

```
[18]: # we need to reshape the images and perform a grayscale to reduce the efect of
      ↪ illumination and perform normalization for all data

X_train = X_train.values.reshape(-1,28,28,1)
test = test.values.reshape(-1,28,28,1)
X_train.shape, test.shape
```

```
[18]: ((42000, 28, 28, 1), (28000, 28, 28, 1))
```

```
[19]: #pip install tensorflow
```

```
[20]: # We will train_test_split

# Convert the lables to one-hot encoding
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
num_classes = 10
y_train = to_categorical(y_train,num_classes = num_classes)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size =
      ↪0.1, random_state=2)
```

```
2024-10-30 02:42:28.375351: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:32]
Could not find cuda drivers on your machine, GPU will not be used.
2024-10-30 02:42:28.379757: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:32]
Could not find cuda drivers on your machine, GPU will not be used.
2024-10-30 02:42:28.391590: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:485] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
2024-10-30 02:42:28.408563: E
external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:8454] Unable to register
cuDNN factory: Attempting to register factory for plugin cuDNN when one has
already been registered
2024-10-30 02:42:28.413262: E
external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1452] Unable to
register cuBLAS factory: Attempting to register factory for plugin cuBLAS when
one has already been registered
2024-10-30 02:42:28.426123: I tensorflow/core/platform/cpu_feature_guard.cc:210]
```

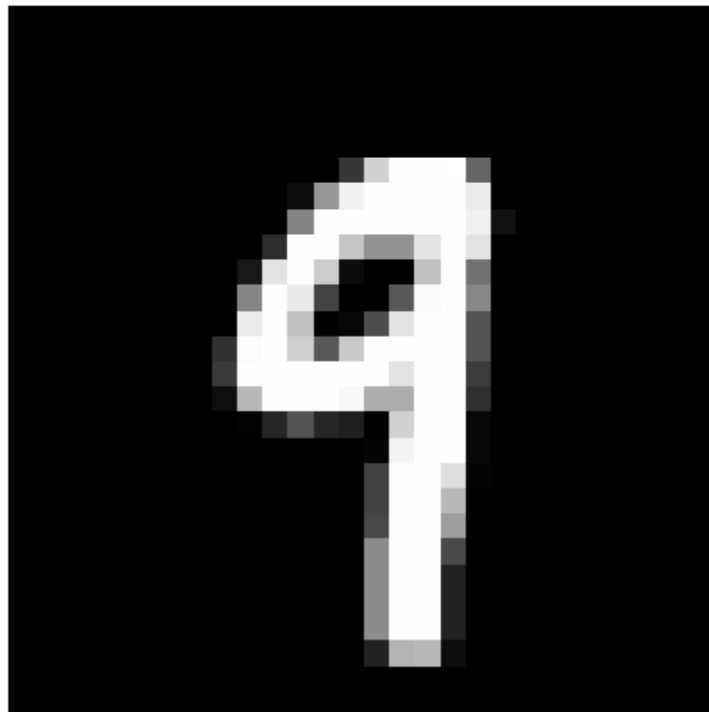
This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2024-10-30 02:42:30.024313: W
tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not find TensorRT

```
[21]: X_train.shape, X_val.shape, y_train.shape, y_val.shape
```

```
[21]: ((37800, 28, 28, 1), (4200, 28, 28, 1), (37800, 10), (4200, 10))
```

```
[22]: plt.imshow(X_train[2][:,:,0], cmap = 'gray')  
plt.title('Sample Image')  
plt.axis('off')  
plt.show()
```

Sample Image



```
[23]: from sklearn.metrics import confusion_matrix , classification_report  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dropout, Dense, Flatten, Conv2D, MaxPool2D  
from tensorflow.keras.optimizers import Adam # Adaptive Moment Estimation  
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

CNN is used for image classification and object detection After having convolution layer we use ReLU to break up linearity. Increase nonlinearity. Because images are non linear.

Padding As we keep applying conv layers, the size of the volume will decrease faster than we would like. In the early layers of our network, we want to preserve as much information about the original input volume so that we can extract those low level features. input size and output size are same

Max Pooling It makes down-sampling or sub-sampling (Reduces the number of parameters) It makes the detection of features invariant to scale or orientation changes. It reduce the amount of parameters and computation in the network, and hence to also control overfitting

Flattening Full Connection Neurons in a fully connected layer have connections to all activations in the previous layer Artificial Neural Netwo

Implementing with Keras¶

Create Model conv => max pool => dropout => conv => max pool => dropout => fully connected (2 layer) Dropout: Dropout is a technique where randomly selected neurons are ignored during trails

```
[24]: # Build the CNN model

model = Sequential()
model.add(Conv2D(filters = 8, kernel_size=(5,5), padding = 'Same', activation = 'relu', input_shape = (28,28,1)))
model.add(MaxPool2D(pool_size=(2,2)))
model.add(Dropout(0.25))

model.add(Conv2D(filters = 16, kernel_size=(3,3), padding = 'Same', activation = 'relu'))
model.add(MaxPool2D(pool_size=(2,2), strides = (2,2)))
model.add(Dropout(0.25))

model.add(Flatten())
model.add(Dense(256, activation = 'relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation = 'softmax'))
```

```
[25]: # Define the optimizer

optimizer = Adam(learning_rate = 0.001, beta_1 = 0.9, beta_2 = 0.999)
model.compile(optimizer=optimizer, loss = 'categorical_crossentropy', metrics = ['accuracy'])
```

Epochs and Batch Size Say you have a dataset of 10 examples (or samples). You have a batch size of 2, and you've specified you want the algorithm to run for 3 epochs. Therefore, in each epoch,

you have 5 batches ($10/2 = 5$). Each batch gets passed through the algorithm, therefore you have 5 iterations per epoch. reference:

<https://stackoverflow.com/questions/4752626/epoch-vs-iteration-when-training-neural-networks>

Data Augmentation To avoid overfitting problem, we need to expand artificially our handwritten digit dataset Alter the training data with small transformations to reproduce the variations of digit. For example, the number is not centered The scale is not the same (some who write with big/small numbers) The image is rotated.

```
[26]: epoch = 10
batch_size = 250

datagen = ImageDataGenerator(featurewise_center = False,
                             samplewise_center = False,
                             featurewise_std_normalization = False,
                             zca_whitening = False,
                             rotation_range = 5,
                             zoom_range = 0.1,
                             width_shift_range = 0.1,
                             height_shift_range = 0.1,
                             horizontal_flip = False,
                             vertical_flip = False)

datagen.fit(X_train)

[27]: history = model.fit(datagen.flow(X_train, y_train,
                                      batch_size = batch_size),
                          epochs = epoch,
                          validation_data = (X_val, y_val),
                          steps_per_epoch = X_train.shape[0]//batch_size)
```

Epoch 1/10

151/151 19s 116ms/step -

accuracy: 0.4721 - loss: 1.5542 - val_accuracy: 0.9376 - val_loss: 0.2185

Epoch 2/10

1/151 12s 85ms/step - accuracy:

0.8360 - loss: 0.4974

2024-10-30 02:42:51.853767: I tensorflow/core/framework/local_rendezvous.cc:404]

Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

[[{{node IteratorGetNext}}]]

151/151 1s 5ms/step -

accuracy: 0.8360 - loss: 0.4974 - val_accuracy: 0.9371 - val_loss: 0.2199

Epoch 3/10

151/151 23s 149ms/step -

accuracy: 0.8458 - loss: 0.4765 - val_accuracy: 0.9621 - val_loss: 0.1340

Epoch 4/10

1/151 12s 83ms/step - accuracy:

0.8720 - loss: 0.3862

```
2024-10-30 02:43:15.804417: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
[[{{node IteratorGetNext}}]]
```

```
151/151          1s 4ms/step -
accuracy: 0.8720 - loss: 0.3862 - val_accuracy: 0.9617 - val_loss: 0.1359
Epoch 5/10
151/151          20s 126ms/step -
accuracy: 0.8996 - loss: 0.3293 - val_accuracy: 0.9669 - val_loss: 0.1113
Epoch 6/10
151/151          1s 4ms/step -
accuracy: 0.9000 - loss: 0.3403 - val_accuracy: 0.9695 - val_loss: 0.1066
Epoch 7/10
151/151          16s 100ms/step -
accuracy: 0.9108 - loss: 0.2803 - val_accuracy: 0.9738 - val_loss: 0.0893
Epoch 8/10
 1/151           8s 59ms/step - accuracy:
0.9440 - loss: 0.1934
```

```
2024-10-30 02:43:52.654657: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
[[{{node IteratorGetNext}}]]
```

```
151/151          1s 4ms/step -
accuracy: 0.9440 - loss: 0.1934 - val_accuracy: 0.9733 - val_loss: 0.0888
Epoch 9/10
151/151          21s 134ms/step -
accuracy: 0.9230 - loss: 0.2470 - val_accuracy: 0.9762 - val_loss: 0.0813
Epoch 10/10
151/151          1s 5ms/step -
accuracy: 0.9280 - loss: 0.2552 - val_accuracy: 0.9755 - val_loss: 0.0820
```

When evaluating a machine learning model, especially a Convolutional Neural Network (CNN) for image classification, there are several key aspects to infer from the validation, training, and error metrics. Here's a breakdown of what to look for and how to interpret the results:

0.0.10 1. Training Loss and Accuracy

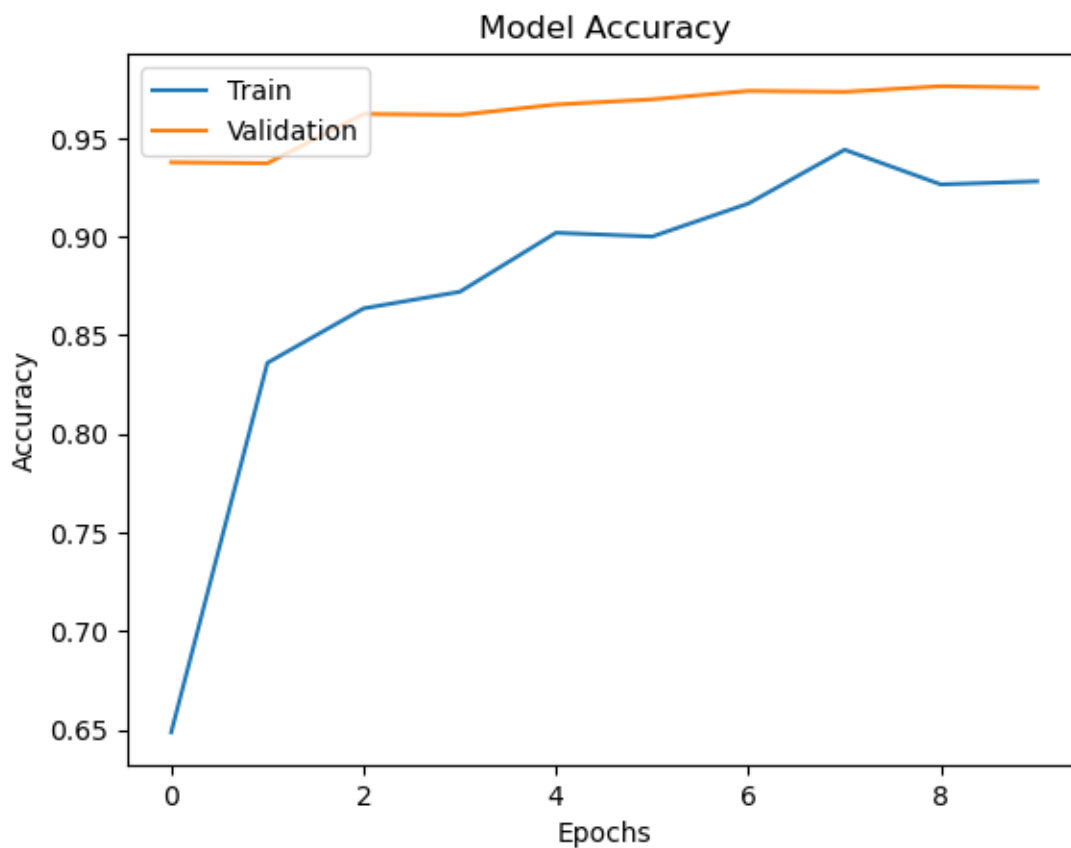
- **Training Loss:** This measures how well the model is performing on the training data. A decreasing training loss indicates that the model is learning from the training data and improving over time. If the training loss is very high, it suggests that the model is not learning effectively or that the learning rate might be too high.
- **Training Accuracy:** This shows the proportion of correctly classified samples in the training set. Increasing training accuracy indicates that the model is getting better at classifying the training samples. A high training accuracy typically means that the model is learning well from the training data.

0.0.11 2. Validation Loss and Accuracy

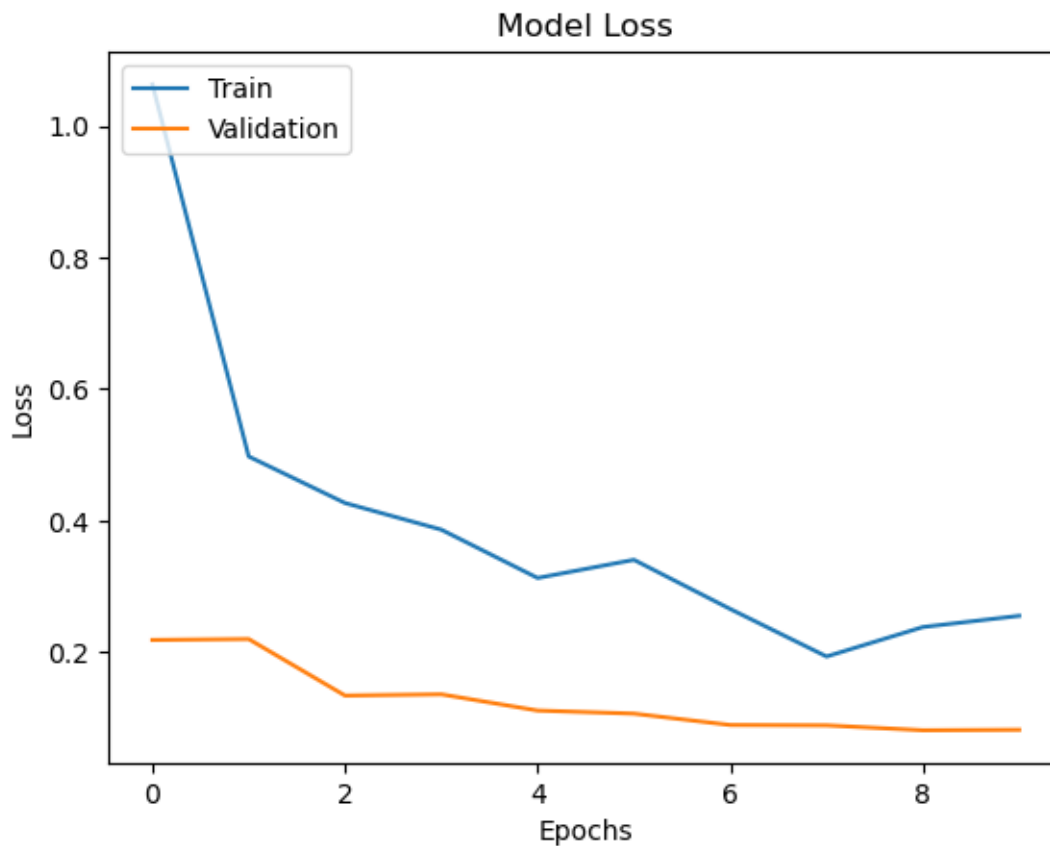
- **Validation Loss:** This measures how well the model performs on unseen validation data. A decreasing validation loss indicates that the model is generalizing well to new, unseen data. If the validation loss starts to increase while the training loss continues to decrease, this is a sign of overfitting.
- **Validation Accuracy:** This shows the proportion of correctly classified samples in the validation set. Increasing validation accuracy indicates that the model is generalizing well to unseen data. If the validation accuracy is much lower than the training accuracy, it suggests overfitting.

```
[28]: # Plot the training and validation accuracy and loss scores
```

```
plt.plot(history.history['accuracy'])  
plt.plot(history.history['val_accuracy'])  
plt.title("Model Accuracy")  
plt.xlabel("Epochs")  
plt.ylabel("Accuracy")  
plt.legend(['Train', 'Validation'], loc = 'upper left')  
plt.show()
```



```
[29]: plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(['Train', 'Validation'], loc = 'upper left')
plt.show()
```

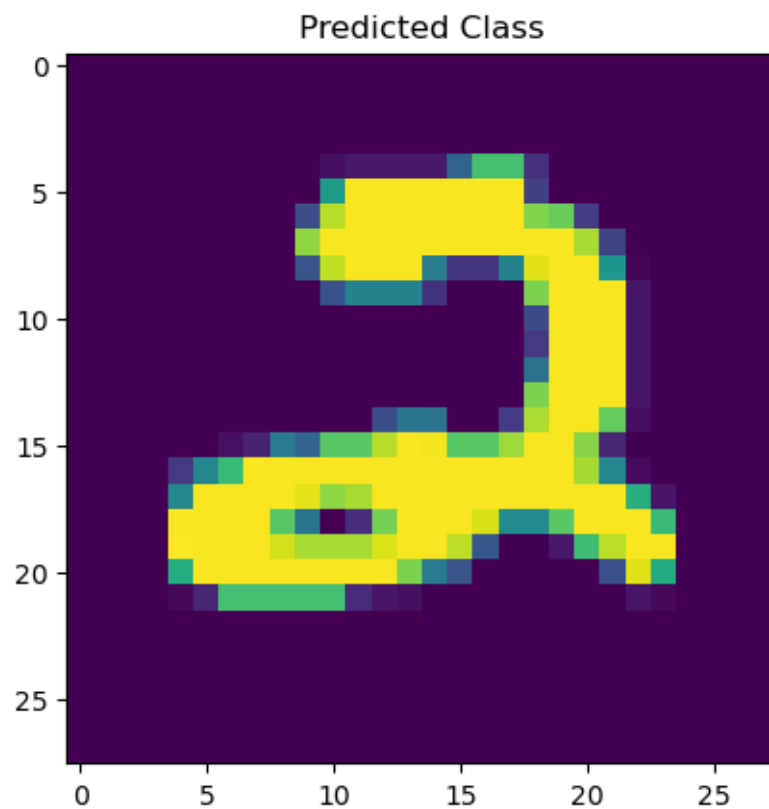


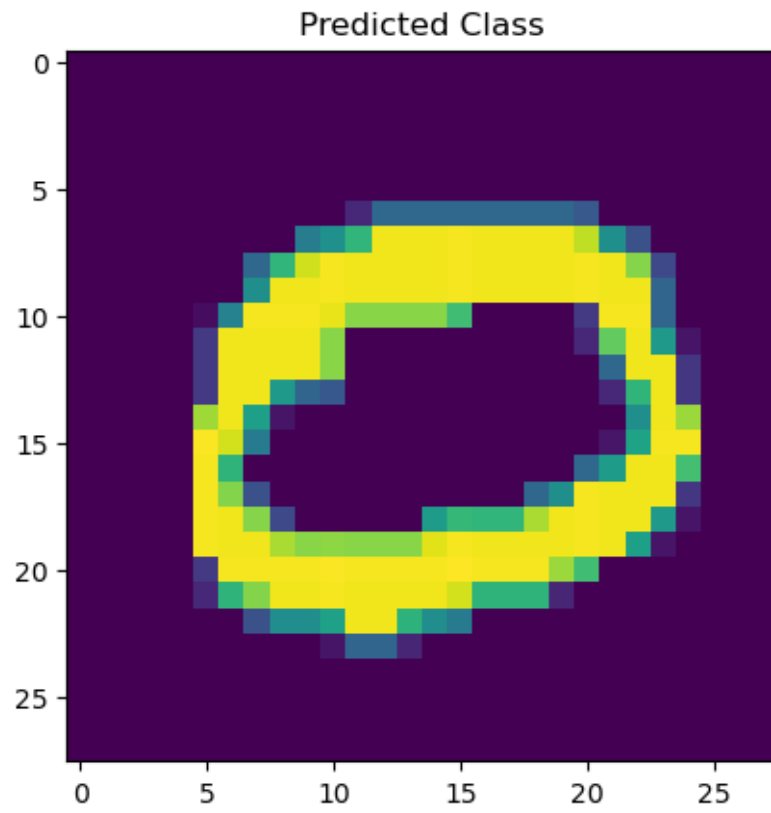
```
[30]: # Testing data
predict = model.predict(test)
print(predict)
```

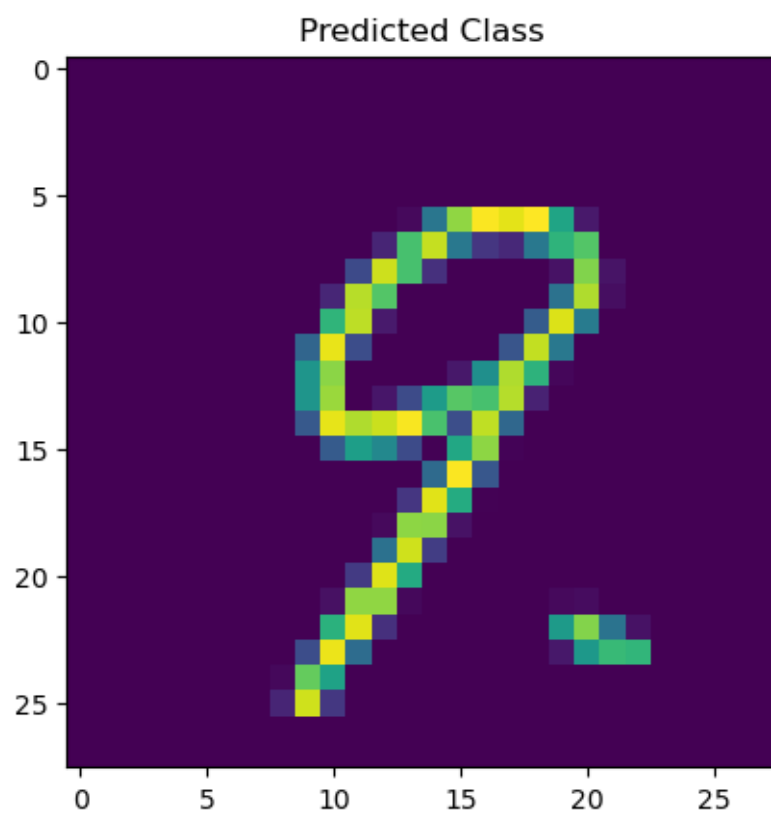
```
875/875          4s 4ms/step
[[1.3022795e-06 3.2154421e-08 9.9956912e-01 ... 3.0990063e-06
 9.1785631e-07 6.5940062e-09]
 [9.9743360e-01 1.5769320e-09 2.6193052e-04 ... 3.5026014e-06
 1.7450214e-04 7.3298259e-04]
 [5.3708383e-04 1.2365617e-03 3.1831358e-02 ... 3.6268965e-03
 1.1400817e-01 8.3568078e-01]
 ...
```

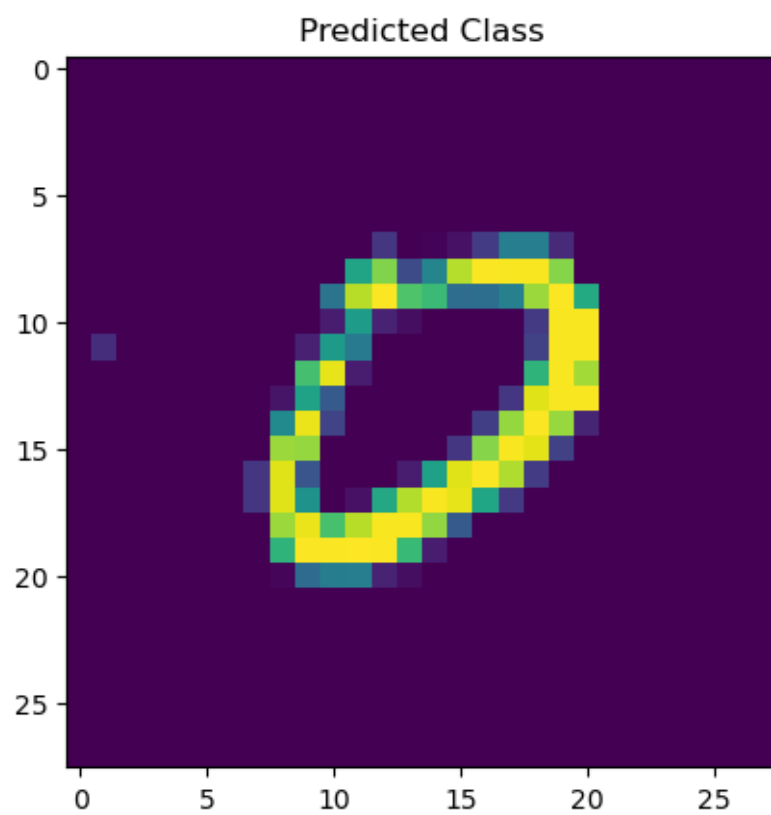
```
[9.6196585e-11 9.2993711e-09 5.3834594e-07 ... 3.3930752e-07  
 5.8776777e-07 2.9991881e-07]  
[1.3017122e-06 1.7426686e-05 3.4301257e-07 ... 2.8709287e-03  
 5.6520486e-05 9.9645215e-01]  
[8.2022228e-07 1.9730791e-08 9.9997860e-01 ... 1.3577294e-07  
 4.7105336e-06 7.7861460e-09]]
```

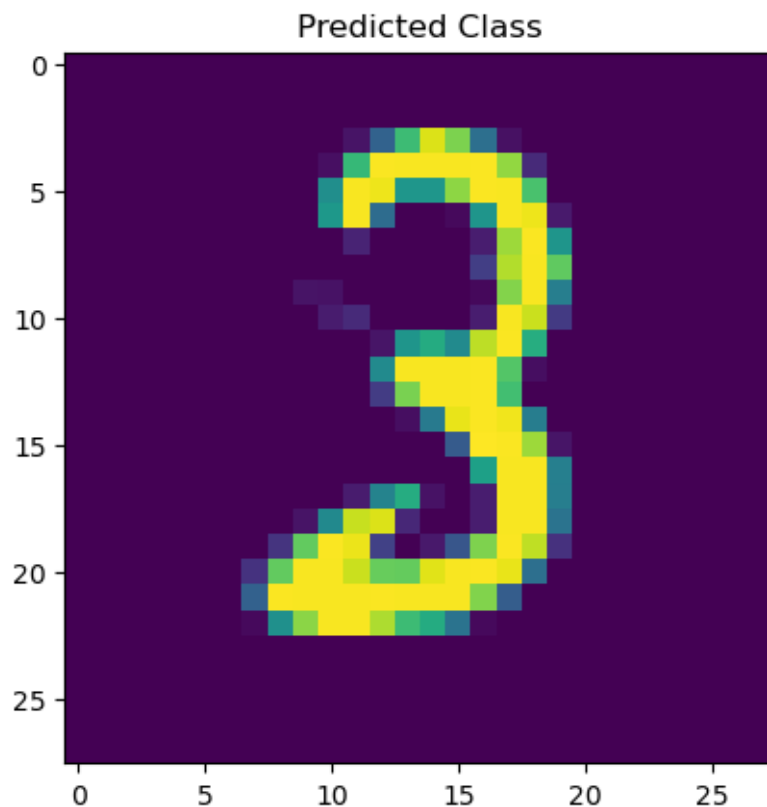
```
[31]: # Plot the predictions  
  
for i in range(5):  
    plt.imshow(test[i])  
    plt.title("Predicted Class")  
    plt.show()
```











```
[75]: # Evaluation Metrics
```

```
predict_labels = np.argmax(predict, axis = 1)
predict_labels
```

```
[75]: array([2, 0, 9, ..., 3, 9, 2])
```

```
[81]: val_predict = model.predict(X_val)
val_predict_labels = np.argmax(val_predict, axis = 1)
y_val_labels = np.argmax(y_val, axis = 1)
```

```
132/132          1s 5ms/step
```

```
[85]: cm = confusion_matrix(y_val_labels, val_predict_labels)
print(cm)
```

```
[[408  0  0  0  1  0  1  0  1  0]
 [  0 473  1  7  1  0  0  0  3  0]
 [  1  0 395  2  0  0  0  0  4  1]
 [  1  0  0 409  0  2  0  1  4  1]
 [  0  2  0  0 441  1  1  1  1 14]
 [  0  0  0  3  0 362  1  0  3  3]]
```

```
[ 4  0  0  0  0  0 404  0  5  0]
[ 0  0  1  0  0  0  0 440  1  4]
[ 0  0  1  1  2  4  1  0 370  3]
[ 1  0  0  5  3  1  0  2  2 395]]
```

```
[87]: classreport = classification_report(y_val_labels, val_predict_labels)
print(classreport)
```

	precision	recall	f1-score	support
0	0.98	0.99	0.99	411
1	1.00	0.98	0.99	485
2	0.99	0.98	0.99	403
3	0.96	0.98	0.97	418
4	0.98	0.96	0.97	461
5	0.98	0.97	0.98	372
6	0.99	0.98	0.98	413
7	0.99	0.99	0.99	446
8	0.94	0.97	0.95	382
9	0.94	0.97	0.95	409
accuracy			0.98	4200
macro avg	0.98	0.98	0.98	4200
weighted avg	0.98	0.98	0.98	4200

0.0.12 3. Overfitting and Underfitting

- **Overfitting:** This occurs when the model performs well on the training data but poorly on the validation data. It typically happens when the model is too complex or trained for too many epochs. Signs include:
 - Training accuracy is high, but validation accuracy is low or stagnating.
 - Training loss decreases, but validation loss starts to increase.
- **Underfitting:** This occurs when the model performs poorly on both the training and validation data. This often happens when the model is too simple or not trained enough. Signs include:
 - Both training and validation accuracy are low.
 - Training and validation loss are high and don't improve over time.

0.0.13 4. Confusion Matrix

- **Confusion Matrix:** This provides a detailed view of how well the model is performing across different classes. It shows:
 - **True Positives (TP):** Correctly classified samples for each class.
 - **False Positives (FP):** Samples incorrectly classified as a given class.
 - **False Negatives (FN):** Samples that belong to a given class but were classified incorrectly.
 - **True Negatives (TN):** Samples correctly classified as not belonging to a given class.

From the confusion matrix, you can infer: - **Class Imbalance:** If certain classes have high false positives or false negatives, it may indicate class imbalance. - **Model Performance:** Which classes the model is performing well on and which it is struggling with. - **Error Types:** Patterns of errors, such as confusion between similar classes.

0.0.14 Interpreting Specific Cases

1. Training and Validation Loss Curves:

- If both curves decrease and stabilize, your model is likely well-trained.
- If training loss decreases while validation loss increases, overfitting might be occurring.
- If both curves are high, your model might be underfitting.

2. Training and Validation Accuracy Curves:

- If both accuracy curves are increasing and eventually plateau, your model is likely learning effectively.
- If training accuracy is high but validation accuracy is low or decreasing, it indicates overfitting.
- If both accuracies are low, consider increasing model complexity or training more epochs.

3. Confusion Matrix:

- Look for high counts in diagonal entries (true positives), which indicates good performance.
- Analyze off-diagonal entries to understand which classes are confused with each other.

0.0.15 Example Interpretation

- **Training Loss:** Decreases from 0.6 to 0.1.
- **Validation Loss:** Decreases from 0.7 to 0.2 but starts to increase after 8 epochs.

Inference: - The model is learning well from the training data, but it begins to overfit after a certain point, as indicated by the increasing validation loss after 8 epochs. - **Action:** You might consider stopping training earlier (early stopping), using regularization techniques, or tuning hyperparameters to improve generalization.

By analyzing these metrics, you can make informed decisions about how to adjust your model, data, and training process to achieve better performance.

[]: